

Laboratorio de rutas y algoritmos de búsqueda

Informe técnico · RA1 PIA · Daniel Magariño Baranda · CEBANC, especialidad IA y Big Data
Repositorio: <https://github.com/daaniel53254/laboratorio-rutas>
Aplicación desplegada: <https://laboratorio-rutas-daniel.netlify.app/>

1. Resumen

Aplicación web con un tablero editable de 20 × 20 en el que se define un inicio, una o varias metas, obstáculos y baldosas con coste. Ejecuta BFS, DFS, búsqueda de coste uniforme (UCS), A* y, como mejora, búsqueda bidireccional, muestra la exploración celda a celda con pausa, paso a paso y control de velocidad, y compara los cinco algoritmos sobre el mismo mapa. Incluye cuatro escenarios reproducibles y 18 pruebas automáticas que se ejecutan con `node tests/run-tests.mjs`. Funciona en localhost (Live Server o `npx serve`) y en Netlify.

2. Elección de lenguaje y herramientas (CE a-d)

Decisión	Elección	Motivo	Alternativa descartada
Lenguaje	JavaScript (módulos ES)	Se ejecuta en el navegador sin instalar nada, lo que permite visualizar el algoritmo paso a paso y desplegar en Netlify como sitio estático.	Python: buen lenguaje para IA, pero necesitaría servidor o Pyodide para la parte visual. En mi equipo Python ni siquiera estaba instalado.
Marcado	HTML5 semántico	Estructura, accesibilidad (roles, etiquetas, <code>aria-live</code>) y sin dependencias.	Canvas: más rápido de dibujar, pero no es accesible ni navegable por teclado.
Framework	Ninguno	Con 2-3 h de trabajo, un proyecto sin build ni dependencias es más fácil de entender, defender y desplegar.	React + TypeScript + Vite: propuesto al inicio, descartado por sobrecoste de configuración.
Estilos	CSS propio	Tema claro/oscuro con variables CSS y diseño adaptable a móvil.	Librerías CSS.
Pruebas	Node + <code>assert</code>	Los algoritmos no dependen del DOM, así que se prueban sin navegador.	Jest/Vitest.
Control de versiones y despliegue	Git, GitHub, Netlify	Entrega verificable; Netlify publica la carpeta raíz sin build.	—

Lenguaje de marcado, estilos y programación

Capa	Archivo	Qué hace
Marcado (HTML)	<code>index.html</code>	Describe el contenido: <code><header></code> con título; <code><main></code> con tres <code><section></code> (editar, ejecutar, comparar) y el tablero; <code><fieldset>/<legend></code> con botones de radio para las herramientas; <code><label></code> asociado a cada <code><input></code> ; <code><dl></code> para las métricas; <code><table></code> con <code><caption></code> y <code><th scope></code> para la comparación;

		<code>role="grid"</code> en el tablero y <code>role="status" / aria-live</code> en los mensajes; enlace «Saltar al tablero».
Estilos (CSS)	<code>css/style.css</code>	Aspecto, sin lógica: cuadrícula de 20 columnas con CSS Grid, colores de cada tipo de celda, foco visible, tema oscuro con <code>prefers-color-scheme</code> y <code>prefers-reduced-motion</code> .
Programación (JavaScript)	<code>js/*.js</code>	Comportamiento: reglas del tablero, algoritmos, eventos y animación. Es lo único que <i>calcula</i> ; HTML y CSS no toman decisiones.

3. Arquitectura

Archivo	Responsabilidad
<code>index.html</code>	Estructura y controles.
<code>css/style.css</code>	Estilos y temas.
<code>js/board.js</code>	Modelo del tablero y reglas de edición (<code>createBoard</code> , <code>place</code> , <code>loadScenario</code>).
<code>js/algorithms.js</code>	<code>bfs</code> , <code>dfs</code> , <code>ucs</code> , <code>astar</code> , <code>bidirectional</code> , cola de prioridad, heurística y <code>validate</code> . No toca el DOM.
<code>js/scenarios.js</code>	Cuatro mapas de texto reproducibles.
<code>js/app.js</code>	Interfaz: pinta el tablero, gestiona eventos, animación, métricas y comparación.
<code>tests/run-tests.mjs</code>	18 pruebas de algoritmos y tablero.
<code>netlify.toml</code>	Publicación de la carpeta raíz sin build.

Estructura de datos del mapa

```
board = {
  size: 20,
  wall: Uint8Array(400) // 1 = obstáculo
  weight: Uint8Array(400) // coste de entrar en la celda (1 por defecto, 2-9 si es baldosa
cara)
  start: índice o -1 // una sola celda de inicio
  goals: Map(índice -> penalización >= 0)
}
índice de celda: i = y * 20 + x
```

Se usan arrays planos indexados por `i = y · 20 + x`: son rápidos, fáciles de copiar y evitan objetos por celda. El estado coherente se garantiza en una sola función: toda edición pasa por `clearCell`, que borra muro, peso, inicio y meta de la celda antes de aplicar la herramienta. Así una celda nunca es muro y meta a la vez, y solo existe un inicio (al colocarlo de nuevo, se mueve).

Flujo desde el clic hasta el resultado

```
Edición: pointerdown / teclado -> apply(i) -> place(board, herramienta, i)
        -> si cambió: se descarta el resultado anterior -> render()
Ejecución: botón Ejecutar -> validate(board) (falta inicio o meta -> mensaje)
        -> ALGORITMOS[algo](board) -> {found, path, explored, steps, cost}
        -> tick(): revela una celda explorada cada N ms (pausa / paso / velocidad)
        -> finish(): pinta la ruta y muestra pasos, coste y exploradas
Comparar: ejecuta los cinco sobre el mismo board -> tabla -> "Ver" repinta un recorrido
```

Los algoritmos se ejecutan completos de una vez y devuelven la lista ordenada de celdas exploradas; la animación solo reproduce esa lista. Esto separa el cálculo de la visualización y permite probar los algoritmos sin navegador.

4. Los algoritmos (CE e)

Reglas comunes. Movimiento ortogonal en orden fijo (arriba, derecha, abajo, izquierda), lo que hace los resultados reproducibles. Entrar en una celda cuesta 1 o su peso. Al llegar a una meta se suma su penalización (≥ 0) al coste total. La ruta se reconstruye siguiendo el array `parent` desde la meta hasta el inicio. El coste de todas las rutas se calcula con la misma función (`pathCost`), por lo que las cifras son comparables.

	BFS	DFS	UCS	A*
Estructura	Cola FIFO	Pila LIFO	Cola de prioridad (montículo binario)	Cola de prioridad (montículo binario)
Criterio de selección	El más antiguo en la cola	El último apilado	Menor coste acumulado g	Menor $f = g + h$
Visitados	Al encolar (<code>parent</code> distinto de -2)	Al desapilar	Cerrado al sacarlo de la cola; se reinserta si se mejora g	Igual que UCS
¿Óptimo?	En pasos, no en coste	No	Sí, en coste (pesos no negativos)	Sí, con heurística admisible y consistente
Límite	Ignora pesos y penalizaciones	Puede dar rutas muy largas; ignora costes	Explora en todas direcciones	Depende de la calidad de h

Metas con penalización en UCS y A*

Si se parara al sacar la primera meta de la cola, una meta cercana con penalización alta ganaría a otra más lejana y barata. Para evitarlo, al sacar una meta se reinserta en la cola una entrada «final» con prioridad $g + \text{penalización}$; el algoritmo termina cuando se saca una entrada final. Así se elige la meta de menor coste total.

Heurística de A*

$h(n) = \text{mínimo sobre las metas de } (\text{distancia Manhattan}(n, \text{meta}) + \text{penalización de esa meta})$. Es admisible porque cada paso cuesta al menos 1 y Manhattan es el mínimo de pasos posible sin diagonales, así que nunca sobreestima. Es consistente ($h(n) \leq \text{coste}(n \rightarrow m) + h(m)$), por lo que cerrar un nodo al sacarlo garantiza que su g es óptimo. Con $h = 0$ el algoritmo es UCS. En caso de empate en f, se prefiere el mayor g, lo que reduce las exploraciones.

Búsqueda bidireccional (mejora)

Ver sección 8. Es un BFS doble: usa dos colas FIFO (una desde el inicio y otra desde todas las metas), marca como visitadas las celdas al encolarlas, reconstruye la ruta uniendo los dos `parent` en el punto de encuentro, e ignora pesos y penalizaciones como BFS.

Casos límite

Sin inicio o sin meta, `validate` devuelve el mensaje y no se ejecuta nada. Si la cola o pila se vacía sin llegar a una meta, se informa «Sin solución» con las celdas exploradas.

5. Escenarios de desafío y comparación

Cada escenario es un mapa de texto en `js/scenarios.js` (# muro, S inicio, G meta, 2-9 peso), por lo que se carga siempre igual.

Escenario	Qué pone a prueba	BFS pasos / coste / expl.	DFS	UCS	A*	Bidir.	Conclusión
1 · Menos pasos	Caminos de distinta longitud, sin pesos	31 / 31 / 341	267 / 267 / 270	31 / 31 / 343	31 / 31 / 32	31 / 31 / 276	BFS, UCS, A* y bidireccional dan el mínimo de pasos. DFS encuentra ruta pero 8,6 veces más larga. A* explora 32 celdas frente a 343.
2 · Atajo caro	Ruta corta cara frente a rodeo barato	17 / 113 / 246	57 / 57 / 58	21 / 21 / 316	21 / 21 / 28	17 / 113 / 155	BFS y la bidireccional minimizan pasos y pagan 113. UCS y A* dan 21, cinco veces menos.
3 · Varias metas	Meta cercana con penalización 30 frente a meta lejana barata	3 / 33 / 17	57 / 57 / 58	21 / 21 / 313	21 / 21 / 41	3 / 33 / 7	BFS y la bidireccional van a la cercana (3 pasos, coste 33). UCS y A* eligen la lejana por coste total (21).
4 · Sin solución	Barrera completa	sin ruta / 180	sin ruta / 180	sin ruta / 180	sin ruta / 180	sin ruta / 206	Los cinco terminan e informan de que no existe ruta. Los cuatro primeros exploran toda la zona alcanzable (180 celdas); la bidireccional explora 206 porque agota también un tramo del lado de las metas.

Nota sobre DFS: en el escenario 2 su coste (57) es menor que el de BFS (113) solo porque su orden de exploración esquiva el pasillo caro, no porque optimice nada. DFS devuelve la primera ruta que encuentra, así que no garantiza ni pasos ni coste mínimos.

1. Editar el tablero

Herramienta

- ☐ S Inicio ☐ G Meta
☒ Obstáculo ☐ S Peso
☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciar tablero

Escenario

1 · Menos pasos

Cargar escenario

Sin pesos. Un bloque con hueco en el centro y campo abierto: hay muchos caminos de distinta longitud. Esperado: BFS, UCS y A* dan los pasos mínimos. DFS puede dar una ruta más larga.

2. Ejecutar

Algoritmo

A*

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad



Exploradas

32

Pasos

31

Coste total

31

A*: ruta encontrada en 31 pasos con coste 31.

3. Comparar los algoritmos

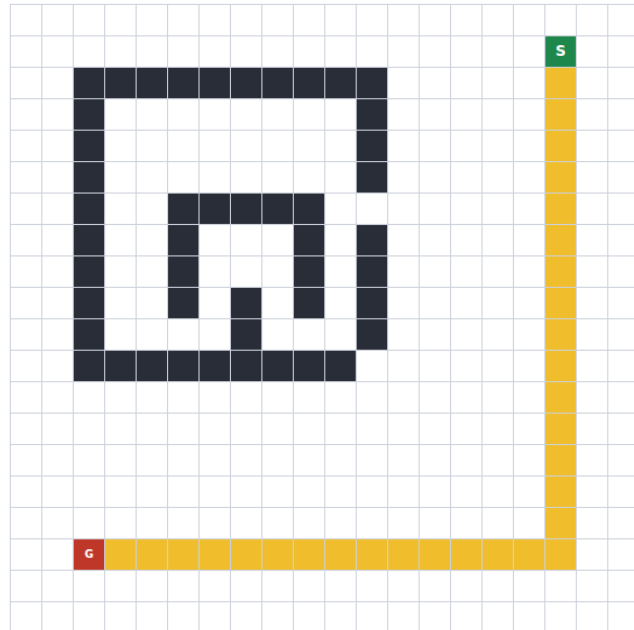
Comparar en este mapa

Algoritmo	Pasos	Coste	Explor.	Ver
BFS	31 ✓	31 ✓	341	<button>Ver</button>
DFS	267	267	270	<button>Ver</button>
UCS	31 ✓	31 ✓	343	<button>Ver</button>
A*	31 ✓	31 ✓	32	<button>Ver</button>
Bidireccional	31 ✓	31 ✓	276	<button>Ver</button>

✓ = mejor valor de la columna. UCS y A* coinciden en coste (31 y 31); A* explora 32 celdas frente a 343 de UCS.

Proyecto RA1 · PIA. HTML define la estructura, CSS los estilos y JavaScript la lógica.

Tablero 20 × 20



S Inicio G Meta (+penalización) Obstáculo S Peso Explorada Ruta final

Escenario 1 con A*. El inicio (S) está arriba a la derecha y la meta (G) abajo a la izquierda, con un bloque de muros en medio. La ruta amarilla rodea el bloque y las celdas azules son las que el algoritmo ha explorado.

Qué demuestra: la heurística Manhattan lleva a A* directo hacia la meta: 31 pasos (el mínimo), coste 31 y solo 32 celdas exploradas, casi todas de la propia ruta. Debajo, la tabla compara los cinco algoritmos sobre este mismo mapa. *Colores: azul claro = celda explorada, amarillo = ruta final, S = inicio, G = meta, marrón con número = baldosa con coste, oscuro = obstáculo.*

1. Editar el tablero

Herramienta

☐ Inicio
☐ Meta
☒ Obstáculo
☐ Peso
☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciador tablero

Escenario

1 · Menos pasos

Cargar escenario

Sin pesos. Un bloque con hueco en el centro y campo abierto: hay muchos caminos de distinta longitud. Esperado: BFS, UCS y A* dan los pasos mínimos. DFS puede dar una ruta más larga.

2. Ejecutar

Algoritmo

DFS (profundidad)

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad

Exploradas

Pasos

Coste total

270

267

267

DFS: ruta encontrada en 267 pasos con coste 267.

Tablero 20 x 20

S Inicio
G Meta (+penalización)
Obstáculo
5 Peso
Explorada
Ruta final

Escenario 1 con DFS. Mismo mapa, pero la ruta amarilla es enorme: 267 pasos. **Qué demuestra:** DFS avanza por un camino hasta que no puede más y solo entonces retrocede, así que recorre casi todo el mapa (270 celdas exploradas). Encuentra una ruta, pero 8,6 veces más larga que la óptima (31 pasos): no garantiza ni pasos ni coste mínimos.

1. Editar el tablero

Herramienta

- ☐ S Inicio ☐ G Meta
☒ ■ Obstáculo ☐ 5 Peso
☐ ▨ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciar tablero

Escenario

2 · Atajo caro ▾

Cargar escenario

Pasillo directo de 12 baldosas de coste 9 frente a un rodeo de 4 pasos más con coste 1. Esperado: BFS toma el pasillo (menos pasos, coste 113). UCS y A* toman el rodeo (coste 21).

2. Ejecutar

Algoritmo

A* ▾

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad



Exploradas

28

Pasos

21

Coste total

21

A*: ruta encontrada en 21 pasos con coste 21.

3. Comparar los algoritmos

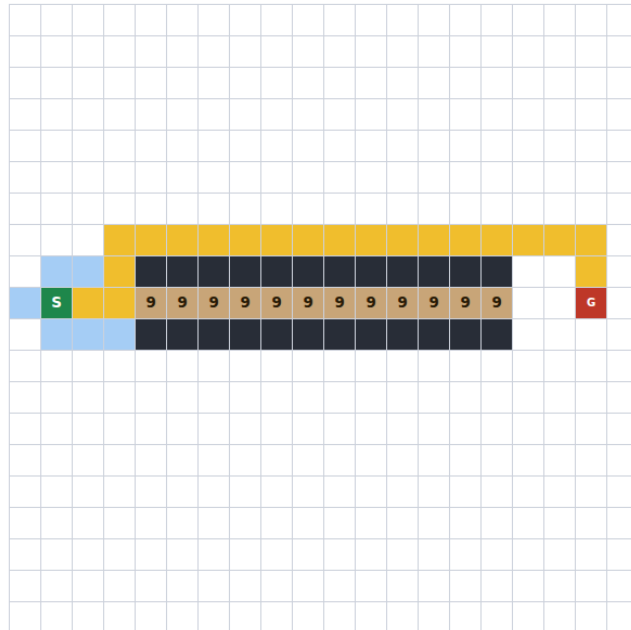
Comparar en este mapa

Algoritmo	Pasos	Coste	Explor.	Ver
BFS	17 ✓	113	246	<button>Ver</button>
DFS	57	57	58	<button>Ver</button>
UCS	21	21 ✓	316	<button>Ver</button>
A*	21	21 ✓	28	<button>Ver</button>
Bidireccional	17 ✓	113	155	<button>Ver</button>

✓ = mejor valor de la columna. UCS y A* coinciden en coste (21 y 21); A* explora 28 celdas frente a 316 de UCS.

Proyecto RA1 · PIA. HTML define la estructura, CSS los estilos y JavaScript la lógica.

Tablero 20 × 20



S Inicio G Meta (+penalización) ■ Obstáculo 5 Peso Explorada Ruta final

Escenario 2 con A*. El inicio está a la izquierda y la meta a la derecha. El pasillo central (12 baldosas marrones de coste 9 entre dos muros) es el camino más corto en pasos, pero muy caro. A* lo evita y rodea por arriba. **Qué demuestra:** A* minimiza el coste, no los pasos: 21 pasos, coste 21 y solo 28 celdas exploradas.

1. Editar el tablero

Herramienta

☐ Inicio
☐ Meta
☒ Obstáculo
☐ Peso
☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciador tablero

Escenario

2 · Atajo caro

Cargar escenario

Pasillo directo de 12 baldosas de coste 9 frente a un rodeo de 4 pasos más con coste 1. Esperado: BFS toma el pasillo (menos pasos, coste 113). UCS y A* toman el rodeo (coste 21).

2. Ejecutar

Algoritmo

BFS (anchura)

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad

Exploradas

Pasos

Coste total

246

17

113

BFS: ruta encontrada en 17 pasos con coste 113.

Tablero 20 × 20

S Inicio

G Meta (+penalización)

Obstáculo

Peso

Explorada

Ruta final

Escenario 2 con BFS. BFS ignora los pesos y elige lo más corto en pasos: cruza el pasillo caro (17 pasos) y paga coste 113, 5,4 veces más que A*. Explora 246 celdas. **Qué demuestra:** minimizar pasos no es lo mismo que minimizar coste.

1. Editar el tablero

Herramienta

- ☐ **S** Inicio ☐ **G** Meta
☒ **■** Obstáculo ☐ **S** Peso
☐ **■** Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciar tablero

Escenario

3 · Varias metas

Cargar escenario

Meta cercana con penalización 30 y meta lejana sin penalización, con un muro que obliga a rodear. Esperado: BFS va a la cercana (3 pasos, coste 33). UCS y A* van a la lejana (coste 21).

2. Ejecutar

Algoritmo

A*

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad



Exploradas

41

Pasos

21

Coste total

21

A*: ruta encontrada en 21 pasos con coste 21.

3. Comparar los algoritmos

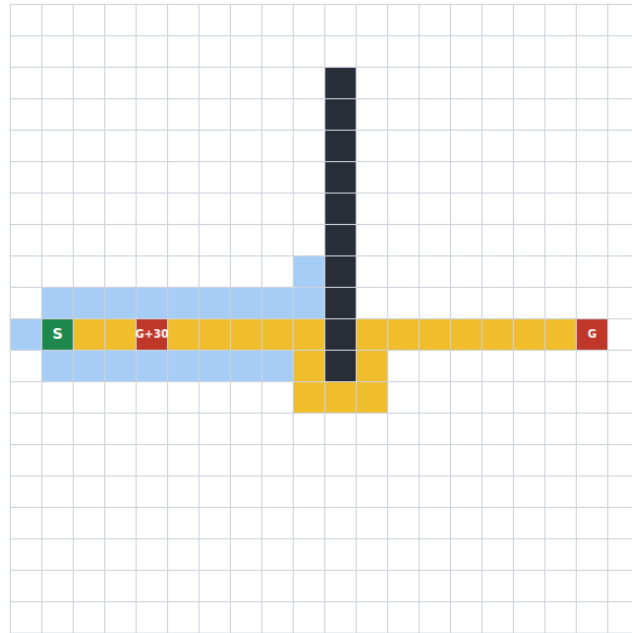
Comparar en este mapa

Algoritmo	Pasos	Coste	Explor.	Ver
BFS	3 ✓	33	17	<button>Ver</button>
DFS	57	57	58	<button>Ver</button>
UCS	21	21 ✓	313	<button>Ver</button>
A*	21	21 ✓	41	<button>Ver</button>
Bidireccional	3 ✓	33	7	<button>Ver</button>

✓ = mejor valor de la columna. UCS y A* coinciden en coste (21 y 21); A* explora 41 celdas frente a 313 de UCS.

Proyecto RA1 · PIA. HTML define la estructura, CSS los estilos y JavaScript la lógica.

Tablero 20 × 20



S Inicio **G** Meta (+penalización) **■** Obstáculo **S** Peso Explorada Ruta final

Escenario 3 con A*. Hay dos metas: la cercana (G+30, a 3 pasos) tiene una penalización de 30 y la lejana (a la derecha, tras un muro) no tiene ninguna. A* elige la lejana: 21 pasos y coste 21, frente a 33 de la cercana (3 pasos + 30). **Qué demuestra:** UCS y A* deciden por coste total, penalización incluida.

1. Editar el tablero

Herramienta

☐ Inicio
☐ Meta
☒ Obstáculo
☐ Peso
☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciador tablero

Escenario

3 · Varias metas

Cargar escenario

Meta cercana con penalización 30 y meta lejana sin penalización, con un muro que obliga a rodear. Esperado: BFS va a la cercana (3 pasos, coste 33). UCS y A* van a la lejana (coste 21).

2. Ejecutar

Algoritmo

BFS (anchura)

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad

Exploradas

Pasos

Coste total

17

3

33

BFS: ruta encontrada en 3 pasos con coste 33.

Tablero 20 x 20

S Inicio

G Meta (+penalización)

Obstáculo

Peso

Explorada

Ruta final

Escenario 3 con BFS. BFS elige la meta cercana por tener menos pasos (3) sin mirar la penalización, y acaba con coste 33, peor que el 21 de A*. Explora solo 17 celdas. **Qué demuestra:** BFS prioriza pasos, no coste.

1. Editar el tablero

Herramienta

- ☐ S Inicio ☐ G Meta
☒ Obstáculo ☐ S Peso
☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciar tablero

Escenario

1 · Menos pasos

Cargar escenario

Sin pesos. Un bloque con hueco en el centro y campo abierto: hay muchos caminos de distinta longitud. Esperado: BFS, UCS y A* dan los pasos mínimos. DFS puede dar una ruta más larga.

2. Ejecutar

Algoritmo

Bidireccional (BFS doble)

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad



Exploradas

276

Pasos

31

Coste total

31

Bidireccional: ruta encontrada en 31 pasos con coste 31.

3. Comparar los algoritmos

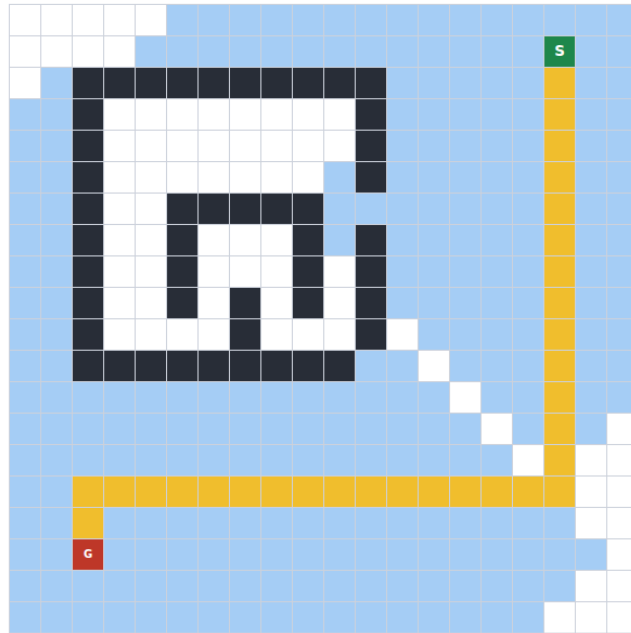
Comparar en este mapa

Algoritmo	Pasos	Coste	Explor.	Ver
BFS	31 ✓	31 ✓	341	<button>Ver</button>
DFS	267	267	270	<button>Ver</button>
UCS	31 ✓	31 ✓	343	<button>Ver</button>
A*	31 ✓	31 ✓	32	<button>Ver</button>
Bidireccional	31 ✓	31 ✓	276	<button>Ver</button>

✓ = mejor valor de la columna. UCS y A* coinciden en coste (31 y 31); A* explora 32 celdas frente a 343 de UCS.

Proyecto RA1 · PIA. HTML define la estructura, CSS los estilos y JavaScript la lógica.

Tablero 20 × 20



S Inicio G Meta (+penalización) Obstáculo S Peso Explorada Ruta final

Escenario 1 con la búsqueda bidireccional. Se lanzan dos búsquedas a la vez, una desde S y otra desde G, y la ruta se forma donde se encuentran. **Qué demuestra:** da los mismos 31 pasos que BFS pero explora 276 celdas en lugar de 341 (un 19 % menos).

1. Editar el tablero

Herramienta

☐ Inicio
☐ Meta
☒ Obstáculo
☐ Peso
☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciartablero

Escenario

3 · Varias metas
Cargar escenario

Meta cercana con penalización 30 y meta lejana sin penalización, con un muro que obliga a rodear. Esperado: BFS va a la cercana (3 pasos, coste 33). UCS y A* van a la lejana (coste 21).

2. Ejecutar

Algoritmo

Bidireccional (BFS doble)
Ejecutar

Pausa

Paso

Quitar resultado

Velocidad

Exploradas

7

Pasos

3

Coste total

33

Bidireccional: ruta encontrada en 3 pasos con coste 33.

3. Comparar los algoritmos

Comparar en este mapa

Algoritmo	Pasos	Coste	Explor.	Ver
BFS	3 ✓	33	17	Ver
DFS	57	57	58	Ver
UCS	21	21 ✓	313	Ver
A*	21	21 ✓	41	Ver
Bidireccional	3 ✓	33	7	Ver

✓ = mejor valor de la columna. UCS y A* coinciden en coste (21 y 21); A* explora 41 celdas frente a 313 de UCS.

Tablero 20 x 20

Proyecto RA1 · PIA. HTML define la estructura, CSS los estilos y JavaScript la lógica.

Escenario 3 con la bidireccional. Parte a la vez del inicio y de las dos metas y se encuentra con la más cercana por pasos: 3 pasos, coste 33 (igual que BFS, porque ignora las penalizaciones). **Qué demuestra:** explora solo 7 celdas frente a las 17 de BFS (un 59 % menos).

1. Editar el tablero

Herramienta

- ☐ Inicio
- ☐ Meta
- ☒ Obstáculo
- ☐ Peso
- ☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciar tablero

Escenario

4 · Sin solución

Cargar escenario

Una barrera vertical completa separa el inicio (izquierda) de las dos metas (derecha). Esperado: Los cuatro algoritmos terminan e informan de que no existe ruta.

2. Ejecutar

Algoritmo

BFS (anchura)

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad

Exploradas

180

Pasos

-

Coste total

-

BFS: Sin solución: no existe ruta a ninguna meta.

Tablero 20 x 20

S Inicio

G Meta (+penalización)

Obstáculo

Peso

Explorada

Ruta final

Escenario 4 con BFS. Una barrera vertical completa separa el inicio (izquierda) de las dos metas (derecha). BFS explora todo el lado izquierdo (180 celdas), no encuentra ruta y la aplicación lo indica en rojo: «Sin solución». **Qué demuestra:** el algoritmo termina y avisa, en lugar de quedarse buscando.

1. Editar el tablero

Herramienta

☐ Inicio

☐ Meta

☒ Obstáculo

☐ Peso

☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciar tablero

Escenario

4 · Sin solución

Cargar escenario

Una barrera vertical completa separa el inicio de las dos metas. Esperado: Los cuatro algoritmos terminan e informan de que no existe ruta.

2. Ejecutar

Algoritmo

Bidireccional (BFS doble)

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad

Exploradas

206

Pasos

-

Coste total

-

Bidireccional: Sin solución: no existe ruta a ninguna meta.

Tablero 20 x 20

S Inicio

G Meta (+penalización)

Obstáculo

5 Peso

Explorada

Ruta final

Escenario 4 con la bidireccional. También termina sin ruta, pero explora 206 celdas: agota el lado izquierdo (180) y además parte del derecho, desde las metas. **Qué demuestra:** es un límite de este algoritmo: en mapas sin solución puede explorar más que BFS.

6. Casos de prueba

Cada caso indica escenario, resultado esperado, resultado obtenido y evidencia. Los resultados «obtenidos» de la columna de interfaz se leyeron de la propia aplicación durante la prueba; los automáticos están en `tests/run-tests.mjs` (18 pruebas superadas).

Caso	Entrada / escenario	Esperado	Obtenido	Evidencia
Ruta simple	Inicio (2,2), meta (10,2), sin obstáculos, BFS	8 pasos, coste 8	8 pasos, coste 8	Fig. T1; prueba «ruta simple»
Sin inicio	Solo una meta, A*	Mensaje «Falta el inicio», nada se ejecuta	«Falta el inicio.»	Fig. T2a
Sin meta	Solo inicio, A*	Mensaje de falta de meta	«Falta al menos una meta.»	Fig. T2b
Varias metas	Escenario 3	BFS 3 pasos/coste 33; UCS y A* coste 21 a la meta lejana	BFS 3/33; A* 21/21	Sección 5; prueba «varias metas»
Pesos	Escenario 2	BFS coste 113; UCS y A* coste 21	BFS 113; A* 21	Sección 5; pruebas «pesos» y «escenario 2»
Obstáculos	Inicio (3,9), meta (15,9), muro vertical en $x=9$ de $y=4$ a $y=14$, BFS	Rodea el muro: 12 pasos directos + 12 de desvío = 24	24 pasos, coste 24	Fig. T5; prueba «obstáculos»
Sin solución	Escenario 4, BFS (y los otros cuatro en las pruebas automáticas)	Todos informan sin ruta	«BFS: Sin solución: no existe ruta a ninguna meta.» (180 exploradas)	Fig. escenario 4; prueba «mapa sin solución»
Edición tras ejecutar	Tras ejecutar BFS en el mapa de obstáculos, se añade un muro en (11,9)	Se descarta el resultado anterior; al ejecutar de nuevo se recalcula	Estado: «Tablero editado: se descartó el resultado anterior.», exploradas 0. Nueva ejecución: 24 pasos, 361 exploradas	Fig. T7; prueba «edición tras ejecutar»
Bidireccional	Escenarios 1, 3 y 4, mapa abierto, varias metas y 5000 mapas aleatorios	Mismos pasos que BFS, sin ruta en el 4, menos celdas exploradas que BFS en mapa abierto	Cumple: 31 pasos en el escenario 1 y 3 pasos en el 3; sin ruta en el 4; en 4980 mapas aleatorios, 0 diferencias de pasos con BFS y rutas válidas	Fig. escenarios 1, 3 y 4 con bidireccional; pruebas «bidireccional» (5)
Coherencia del tablero	Poner muro sobre meta, inicio sobre muro, etc.	Una celda nunca tiene dos tipos	Correcto	Prueba «coherencia»
UCS = A* en coste	4 escenarios y 200 mapas aleatorios	Mismo coste en todos	Coinciden en todos	Prueba «UCS y A* coinciden»

con pesos y penalizaciones

1. Editar el tablero

Herramienta

☐ Inicio ☒ Meta

☐ Obstáculo ☐ 5 Peso

☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciar tablero

Escenario

4 · Sin solución

Una barrera vertical completa separa el inicio de las dos metas. Esperado: Los cuatro algoritmos terminan e informan de que no existe ruta.

2. Ejecutar

Algoritmo

BFS (anchura)

Velocidad

Exploradas	Pasos	Coste total
69	8	8

BFS: ruta encontrada en 8 pasos con coste 8.

Tablero 20 × 20

S Inicio G Meta (+penalización) black square Obstáculo 5 Peso blue square Explorada yellow square Ruta final

T1 · Ruta simple con BFS. Inicio en (2,2) y meta en (10,2), sin obstáculos. La ruta amarilla es una línea recta de 8 pasos con coste 8, y la zona azul crece alrededor del inicio en forma de rombo. **Resultado:** coincide con lo esperado (8 pasos).

1. Editar el tablero

Herramienta

☐ Inicio

☒ Meta

☐ Obstáculo

☐ Peso

☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciar tablero

Escenario

4 · Sin solución

Cargar escenario

Una barrera vertical completa separa el inicio de las dos metas. Esperado: Los cuatro algoritmos terminan e informan de que no existe ruta.

2. Ejecutar

Algoritmo

A*

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad

Exploradas

0

Pasos

-

Coste total

-

Falta el inicio.

3. Comparar los algoritmos

Tablero 20 × 20

S Inicio

G Meta (+penalización)

Obstáculo

S Peso

Explorada

Ruta final

T2a · Sin inicio. El tablero solo tiene una meta. Al pulsar Ejecutar, la aplicación muestra en rojo «Falta el inicio.» y no ejecuta ningún algoritmo (0 celdas exploradas). **Resultado:** el caso límite se controla sin errores.

1. Editar el tablero

Herramienta

☒ Inicio
☐ Meta
☐ Obstáculo
☐ Peso
☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciar tablero

Escenario

4 · Sin solución

Cargar escenario

Una barrera vertical completa separa el inicio de las dos metas. Esperado: Los cuatro algoritmos terminan e informan de que no existe ruta.

2. Ejecutar

Algoritmo

A*

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad

Exploradas

Pasos

Coste total

0

-

-

Falta al menos una meta.

3. Comparar los algoritmos

Tablero 20 × 20

S

S

 Inicio

G

 Meta (+penalización)
 Obstáculo

S

 Peso
 Explorada
 Ruta final

T2b · Sin meta. El tablero solo tiene inicio. La aplicación avisa con «Falta al menos una meta.» y no ejecuta nada. **Resultado:** igual que T2a, el caso límite se controla sin errores.

1. Editar el tablero

Herramienta

☐ Inicio
☐ Meta
☒ Obstáculo
☐ Peso
☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciador tablero

Escenario

4 · Sin solución

Cargar escenario

Una barrera vertical completa separa el inicio de las dos metas. Esperado: Los cuatro algoritmos terminan e informan de que no existe ruta.

2. Ejecutar

Algoritmo

BFS (anchura)

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad

Exploradas

Pasos

Coste total

362

24

24

BFS: ruta encontrada en 24 pasos con coste 24.

Tablero 20 × 20

S Inicio
G Meta (+penalización)
Obstáculo
5 Peso
Explorada
Ruta final

T5 · Obstáculos. Un muro vertical separa el inicio (3,9) de la meta (15,9). BFS lo rodea por un extremo: 24 pasos (12 en línea recta más 12 de desvío) y 362 celdas exploradas. **Resultado:** la ruta nunca atraviesa un muro, como se esperaba.

1. Editar el tablero

Herramienta

☐ Inicio
☐ Meta
☒ Obstáculo
☐ Peso
☐ Borrar

Peso de la baldosa (2-9)

5

Penalización de la meta (≥ 0)

0

Vaciador tablero

Escenario

4 · Sin solución

Cargar escenario

Una barrera vertical completa separa el inicio de las dos metas. Esperado: Los cuatro algoritmos terminan e informan de que no existe ruta.

2. Ejecutar

Algoritmo

BFS (anchura)

Ejecutar

Pausa

Paso

Quitar resultado

Velocidad

Exploradas

Pasos

Coste total

0

-

-

Tablero editado: se descartó el resultado anterior.

Tablero 20 × 20

☒ Inicio
☒ Meta (+penalización)
☒ Obstáculo
☒ Peso
☐ Explorada
☐ Ruta final

T7 · Edición tras ejecutar. Después de ejecutar BFS se añadió un obstáculo en (11,9). La aplicación descartó al instante el recorrido anterior (desaparecen las celdas azules y la ruta, y las métricas vuelven a 0) y avisa: «Tablero editado: se descartó el resultado anterior.» **Resultado:** nunca se muestra un resultado desactualizado; al ejecutar de nuevo se recalcula (24 pasos, 361 exploradas).

7. Paso a paso de construcción

1. **Análisis del enunciado.** Identifiqué los requisitos mínimos: tablero 20 × 20, cuatro algoritmos, comparación, cuatro escenarios, pruebas, informe y despliegue.
2. **Decisión de tecnología.** El primer plan usaba React + TypeScript + Vite. Con solo 2-3 horas lo cambié por HTML + JavaScript puro: sin build ni dependencias, más fácil de entender y de desplegar.
3. **Modelo del tablero (board.js).** Decidí centralizar las reglas de edición en `clearCell` para evitar estados incoherentes.
4. **Algoritmos (algorithms.js).** Primero BFS y DFS, después UCS y A* con montículo binario. Añadí la entrada «final» para tratar bien las penalizaciones de meta.
5. **Pruebas antes de la interfaz.** Escribí las pruebas para validar la lógica sin navegador.
6. **Interfaz (app.js , index.html , style.css).** Tablero con botones (accesible por teclado), herramientas, animación con pausa, paso y velocidad, métricas y tabla comparativa.
7. **Despliegue.** Repositorio en GitHub y sitio en Netlify sin build.
8. **Mejora extra.** Búsqueda bidireccional, implementada con ayuda de Claude Code en VS Code, revisada y probada (sección 8).

Dificultades y correcciones

Problema	Cómo lo detecté	Corrección
Una de mis pruebas esperaba 8 pasos al rodear un muro de 5 celdas.	Falló la prueba automática.	Comprobé a mano el camino: son 10 (5 para rodear + 5 de vuelta). El error estaba en la prueba, no en el algoritmo.
En el escenario 1, DFS daba los mismos 38 pasos que BFS por casualidad, y no demostraba nada.	La tabla de resultados de las pruebas.	Recoloqué inicio y meta para que el orden de vecinos de DFS le desvíe: ahora DFS da 267 pasos frente a 31.
La columna «Algoritmo» de la tabla comparativa aparecía cortada, y volvió a ocurrir al añadir «Bidireccional», un nombre más largo.	Al revisar capturas de pantalla.	Reduje tamaño y márgenes, acorté la cabecera «Explorada» a «Explor.» y evité el salto de línea en la primera columna. Comprobé que ya no hay desbordamiento a 1280 y 390 píxeles de ancho.
Python no estaba instalado en mi equipo, así que <code>python3 -m http.server</code> falló.	Mensaje del terminal.	Usé la extensión Live Server de VS Code.
Comando <code>git remote add origin</code> sin URL.	Error de uso del comando.	Ejecutarlo con la URL en la misma línea.
Claude Code no pudo ejecutar las pruebas porque Node.js no estaba instalado.	Mensaje de Claude Code: no localizaba <code>node</code> .	Instalé Node.js LTS, reinicié VS Code y las pruebas pasaron (13, luego 18).
Netlify no encontraba el repositorio.	La lista de repositorios aparecía vacía.	Conceder a la app de Netlify acceso al repositorio en la configuración de GitHub.

8. Mejora implementada: búsqueda bidireccional

Qué es. Dos BFS a la vez: uno sale del inicio y otro sale de todas las metas simultáneamente. Se ejecutan por capas completas y, en cada paso, se expande la frontera más pequeña. Cuando una capa toca celdas ya alcanzadas por el otro lado, se elige el punto de encuentro con menos pasos totales y se unen los dos trozos de ruta. Como el segundo BFS parte de todas las metas, con varias metas la ruta acaba en la más cercana por pasos, sin una búsqueda previa para elegirla.

Por qué ayuda. Un BFS explora un círculo cuyo radio es la distancia total; dos BFS exploran dos círculos de la mitad de radio, con menos área en total. Es la mejora habitual de BFS cuando se conoce la meta.

Caso de uso. Encontrar la ruta con menos pasos entre dos puntos concretos, sin pesos, explorando menos mapa.

Escenario	BFS (exploradas)	Bidireccional (exploradas)	Pasos BFS / bidir.
1 · Menos pasos	341	276	31 / 31
2 · Atajo caro	246	155	17 / 17 (coste 113 en ambos)
3 · Varias metas	17	7	3 / 3
4 · Sin solución	180	206	sin ruta / sin ruta

Efecto medible. Explora menos celdas que BFS en los escenarios 1, 2 y 3 (entre un 19 % y un 59 % menos) con los mismos pasos. En una prueba con 4980 mapas aleatorios (muros del 0 al 45 %, de una a tres metas), dio siempre los mismos pasos que BFS, con rutas válidas, y exploró menos celdas en 4482 mapas y más en 301; en el resto, las mismas.

Pruebas. Cinco pruebas automáticas: mismos pasos que BFS en el escenario 1, sin ruta en el 4, menos celdas exploradas que BFS en un mapa abierto, varias metas (usa la más cercana por pasos aunque tenga penalización) y mismos pasos que BFS en 300 mapas aleatorios.

Límites. (1) Ignora pesos y penalizaciones: solo es óptima en número de pasos, no en coste (en el escenario 2 paga 113 frente a 21 de UCS y A*). (2) En mapas sin solución puede explorar más que BFS (206 frente a 180 en el escenario 4), porque agota también un tramo del lado de las metas. (3) Necesita que el movimiento sea reversible, algo que se cumple en este tablero pero no siempre en otros problemas.

9. Limitaciones

- Sin movimiento diagonal.
- La bidireccional ignora pesos y penalizaciones; solo es óptima en pasos.
- La cola de prioridad y los mapas están pensados para 20 × 20, no se han medido tableros mayores.
- La animación reproduce el recorrido ya calculado; no ejecuta el algoritmo paso a paso internamente.
- Los pesos están limitados a 2-9 y las penalizaciones a números enteros.
- El resultado de DFS depende del orden fijo de vecinos (arriba, derecha, abajo, izquierda).

- Las pruebas visuales se hicieron en Chromium y Brave; no se han probado en Safari ni Firefox.

10. Uso de asistentes de código IA

Herramienta: Claude (Anthropic): en la web/aplicación para la versión inicial y el informe, y Claude Code en VS Code para la mejora bidireccional. No se incluyeron claves, contraseñas ni datos personales en las consultas.

Consulta representativa	Qué propuso	Qué acepté o cambié	Cómo lo verifiqué
«Ayúdame con este trabajo» (enunciado adjunto) y «necesito hacerlo en menos tiempo, tengo 2-3 h»	Un plan en 10 fases con React + TypeScript + Vite y canvas.	Cambié a HTML + JavaScript sin framework por el tiempo disponible. Descarté el canvas por accesibilidad.	Comprobé que el proyecto no necesita instalación y funciona en Live Server y Netlify.
Implementación de BFS, DFS, UCS y A*	Código de los cuatro algoritmos, montículo binario y heurística Manhattan.	Acepté la estructura. Revisé el tratamiento de la penalización de meta (entrada «final») y la consistencia de la heurística.	18 pruebas automáticas, incluida la comparación UCS = A* en 200 mapas aleatorios.
Diseño de los cuatro escenarios	Cuatro mapas de texto.	Se rediseñó el escenario 1 porque DFS coincidía con BFS por casualidad.	Tabla de resultados por escenario y pruebas específicas.
Interfaz y accesibilidad	Tablero de botones con navegación por flechas, roles ARIA y tema claro/oscuro.	Aceptado. Se corrigió la tabla comparativa cortada.	Capturas en navegador y prueba de teclado.
«Añade búsqueda bidireccional» (con plan previo de 3 líneas)	Un plan con dos BFS por capas completas, uno desde el inicio y otro desde todas las metas, y una implementación con 5 pruebas y documentación en el README.	Acepté el plan tras revisarlo. Comprobé que expande la frontera más pequeña en lugar de alternar capas (variante correcta) y que se añadió al final de ALGORITHMS para no romper la nota de la tabla.	18 pruebas; prueba adicional con 5000 mapas aleatorios frente a BFS (0 diferencias); capturas de los escenarios.
Errores de Git y Netlify	Explicación del fallo y comandos correctos.	Aplicados tal cual.	El repositorio y el despliegue funcionan.

11. Enlaces

- Repositorio: <https://github.com/daaniel53254/laboratorio-rutas>
- Aplicación en Netlify: <https://laboratorio-rutas-daniel.netlify.app/>